UNIT V: Virtual Functions, Polymorphism & Exception Handling

Complete Answers: Debugging and Pseudo-Code

Section A: Debugging Questions

Question 1: Non-Virtual Destructor Memory Leak in Polymorphic Deletion

Topic: Virtual Functions, Virtual Destructors, Dynamic Dispatch

1. Why ~ComplexPolygon() is not called

Shape has a virtual render() function, but its destructor is not virtual. Therefore, deleting a ComplexPolygon object through a Shape* does not provide the required polymorphic destruction. The program cannot safely dispatch the destructor chain through the base pointer. The derived destructor, which contains delete[] vertexArray, is not called; the allocated array is consequently leaked. The compiler may select the non-virtual base destructor based on the static type Shape*.

2. C++ rule and classification

If a delete-expression is applied to a pointer to a base class subobject and the static type is not similar to the dynamic type, the base class must have a virtual destructor. Otherwise, the behavior is undefined. This is classified as undefined behavior, not merely a guaranteed skipped function call. The practical symptom in this example is that the derived resource cleanup may not occur.

3. Correct location and vtable operation

Declare the base destructor as virtual: virtual ~Shape(). A virtual destructor participates in dynamic dispatch. The object's vptr refers to the virtual-function table associated with ComplexPolygon, so deletion through Shape* invokes the derived destructor first. After its body releases vertexArray, the Shape destructor runs automatically.

4. Complete corrected code

```
#include <iostream>

class Shape {
public:
    Shape() { std::cout << "Shape created." << std::endl; }
    virtual ~Shape() {
        std::cout << "Shape destroyed." << std::endl;
    }
    virtual void render() const {
        std::cout << "Rendering generic shape." << std::endl;
    }
};

class ComplexPolygon : public Shape {
private:
    double* vertexArray;
    int vertexCount;
public:
    ComplexPolygon(int count) : vertexCount(count) {
        vertexArray = new double[vertexCount * 2];
        std::cout << "Allocated " << vertexCount
                  << " vertices on heap." << std::endl;
    }
    ~ComplexPolygon() override {
        std::cout << "Deallocating polygon vertex heap memory."
                  << std::endl;
        delete[] vertexArray;
    }
    void render() const override {
        std::cout << "Rendering polygon with " << vertexCount
                  << " vertices." << std::endl;
    }
};

int main() {
    Shape* shapePtr = new ComplexPolygon(1000);
    shapePtr->render();
    delete shapePtr;
    return 0;
}
```

Dry run and terminal output

Step	Operation	Result / state
1	new ComplexPolygon(1000)	Shape constructor runs; derived array is allocated.

Step	Operation	Result / state
2	shapePtr->render()	Virtual dispatch calls ComplexPolygon::render().
3	delete shapePtr	ComplexPolygon destructor runs first and releases the array.
4	Destructor chaining	Shape destructor runs after the derived destructor.

```
Shape created.
Allocated 1000 vertices on heap.
Rendering polygon with 1000 vertices.
Deallocating polygon vertex heap memory.
Shape destroyed.
```

Question 2: Inverted Exception Catch Handlers & Unreachable Code

Topic: Exception Handling Mechanism, Catch Handler Ordering

1. Compiler warning

The compiler typically warns that the InsufficientFundsException handler is unreachable because the earlier BankingException handler can already catch it. Exact warning wording depends on the compiler. The derived exception is caught before execution can reach Line 38.

2. Catch matching

C++ checks catch handlers in source order. A handler taking const BankingException& can bind to an object of BankingException or to an object of a class derived from BankingException. Therefore, the thrown InsufficientFundsException matches the base-class reference handler. The later derived handler is shadowed.

3. Ordering rule

Place handlers for the most specific derived exception types before handlers for their base classes. Place a catch-all handler, catch(...), last because it can match any exception.

4. Corrected try-catch block

```
int main() {
    try {
        processWithdrawal(500.0, 750.0);
    }
    catch (const InsufficientFundsException& ife) {
        std::cout << "Shortfall of: $"
                  << ife.getShortfall() << std::endl;
    }
    catch (const BankingException& be) {
        std::cout << "General Banking Error: "
                  << be.getMessage() << std::endl;
    }
    catch (...) {
        std::cout << "Unknown banking error." << std::endl;
    }
    return 0;
}
```

Dry run and terminal output

Step	Operation	Result / state
1	processWithdrawal(500, 750)	750 is greater than 500; shortfall is 250.
2	throw InsufficientFundsException	Control leaves the function and enters catch matching.
3	First handler checked	Derived handler matches and executes.
4	Remaining handlers	Base and catch-all handlers are skipped.

```
Shortfall of: $250
```

Question 3: Exception Rethrowing Slicing Bug (throw e; vs. throw;)

Topic: Throwing, Catching & Rethrowing an Exception, Object Slicing

1. Current output and reason

The audit message is printed first. The statement throw e; creates a new exception object from the expression e, whose static type is DatabaseError. The TimeoutError-specific information (including timeoutMs) is sliced away. In main(), catch(const TimeoutError&) does not match the new DatabaseError object, so the generic DatabaseError handler executes.

2. Difference between throw e; and bare throw;

throw e; throws a new exception object initialized from e. Since e is declared as const DatabaseError&, the new object has DatabaseError type and the derived part is lost. Bare throw; rethrows the currently handled exception object itself, preserving its original dynamic type and derived attributes.

3. Corrected line 32

```
throw; // Rethrow the original TimeoutError object
```

4. Corrected output

```
[AUDIT LOG]: Intercepted DB error.
Client caught TimeoutError! Duration: 5000 ms
```

The catch in intermediateAuditor logs the error, then bare throw transfers the original TimeoutError to main().

Question 4: Attempting to Instantiate an Incomplete Abstract Class

Topic: Pure Virtual Functions, Abstract Classes, Runtime Interfaces

1. Compilation error

The compiler reports that an object of abstract class type VideoPacket cannot be declared because the pure virtual function NetworkPacket::getPacketSize() has no final overrider in VideoPacket. The exact diagnostic wording varies by compiler.

2. Abstract class condition

A class is abstract if it has at least one pure virtual function that has not been overridden by a final overrider. An abstract class cannot be directly instantiated because it defines an incomplete interface.

3. When a derived class becomes concrete

A derived class becomes concrete when it provides implementations for all inherited pure virtual functions that remain unimplemented. It can then be instantiated.

4. Corrected code

```
#include <iostream>

class NetworkPacket {
public:
    virtual ~NetworkPacket() {}
    virtual void serialize() = 0;
    virtual int getPacketSize() = 0;
};

class VideoPacket : public NetworkPacket {
private:
    int frameID;
public:
    VideoPacket(int id) : frameID(id) {}
    void serialize() override {
        std::cout << "Serializing H.264 Video Frame: "
                  << frameID << std::endl;
    }
    int getPacketSize() override {
        return 1024;
    }
};

int main() {
    VideoPacket packet(1042);
    packet.serialize();
    std::cout << "Packet size: " << packet.getPacketSize()
              << " bytes" << std::endl;
    return 0;
}
```

Dry run and terminal output

```
Serializing H.264 Video Frame: 1042
Packet size: 1024 bytes
```

Question 5: Pass-by-Value Object Slicing Suppressing Dynamic Binding

Topic: Runtime Polymorphism, Early vs. Late Binding, Object Slicing

1. Why the generic attack is printed

triggerEncounter accepts Entity e by value. Passing boss copies only the Entity base subobject into e. The Dragon-specific part is sliced off. The copied object is therefore an Entity object, and the virtual call e.attack() dispatches to Entity::attack().

2. Role of the vptr

A polymorphic object uses a vptr to access its class's virtual-function table. When a derived object is passed by reference, the original Dragon object and its dynamic type are preserved, so virtual dispatch selects Dragon::attack(). In the by-value parameter, the new object is an Entity copy; its vptr corresponds to Entity, not Dragon. Thus the derived override is no longer available.

3. Refactored function

```
void triggerEncounter(const Entity& e) {
    e.attack();
}
```

A reference avoids copying and slicing. A base pointer parameter, Entity*, would also preserve dynamic dispatch when it points to the derived object.

4. Corrected complete call and output

```
int main() {
    Dragon boss;
    triggerEncounter(boss);
    return 0;
}
```

```
Dragon breathes scorching fire!
```

Section B: Pseudo-Code Writing Questions

Question 1: Universal Payment Gateway with Runtime Polymorphism

Topic: Pure Virtual Functions, Runtime Polymorphism, Abstract Base Classes

1. Formal structured pseudo-code

```
ABSTRACT CLASS PaymentGateway
  PUBLIC:
    PURE VIRTUAL METHOD Authorize(amount AS REAL)
      RETURNS BOOLEAN
    PURE VIRTUAL METHOD Capture() RETURNS BOOLEAN
    VIRTUAL DESTRUCTOR()
END CLASS

CLASS CreditCardGateway INHERITS PaymentGateway
  PRIVATE:
    cardNumber AS STRING
    cvv AS STRING
  PUBLIC:
    OVERRIDE METHOD Authorize(amount AS REAL) RETURNS BOOLEAN
      IF LENGTH(cardNumber) = 16 AND LENGTH(cvv) = 3
         RETURN TRUE
      ELSE
         RETURN FALSE
      END IF
    END METHOD
    OVERRIDE METHOD Capture() RETURNS BOOLEAN
      OUTPUT "Credit card payment captured."
      RETURN TRUE
    END METHOD
END CLASS

CLASS CryptoGateway INHERITS PaymentGateway
  PRIVATE:
    walletHash AS STRING
  PUBLIC:
    OVERRIDE METHOD Authorize(amount AS REAL) RETURNS BOOLEAN
      IF STARTS_WITH(walletHash, "0x") AND
         amount >= ESTIMATED_GAS_FEE THEN
         OUTPUT "Crypto wallet and gas fee validated."
         RETURN TRUE
      END IF
      RETURN FALSE
    END METHOD
    OVERRIDE METHOD Capture() RETURNS BOOLEAN
      OUTPUT "Cryptocurrency transfer captured."
      RETURN TRUE
    END METHOD
END CLASS

MAIN
```

```
  DECLARE gatewayPtr AS PaymentGateway*
  gatewayPtr <- NEW CryptoGateway("0xA91F...")
  IF gatewayPtr->Authorize(1500.00) = TRUE THEN
    IF gatewayPtr->Capture() = TRUE THEN
      OUTPUT "Payment successful: $1500.00"
    END IF
  ELSE
    OUTPUT "Payment authorization failed."
  END IF
  DEALLOCATE gatewayPtr
END MAIN
```

2. Trace for a $1,500 cryptocurrency payment

Step	Operation	Result / state
1	Create CryptoGateway	gatewayPtr refers to a CryptoGateway object.
2	Authorize(1500.00)	Base pointer dynamically dispatches to CryptoGateway::Authorize.
3	Wallet check	Wallet hash begins with 0x; gas-fee condition is checked.
4	Capture()	Base pointer dispatches to CryptoGateway::Capture.
5	Deallocate gatewayPtr	Virtual destructor runs and safely destroys the derived object.

Assumption for this trace: the supplied wallet hash has the required 0x prefix and the $1,500 amount satisfies the simulated gas-fee check.

```
Crypto wallet and gas fee validated.
Cryptocurrency transfer captured.
Payment successful: $1500.00
```

Question 2: Resilient Banking Engine with Nested Try-Catch & Rethrowing

Topic: Custom Exception Hierarchy, Nested TRY-CATCH, Rethrowing

1. Structured pseudo-code

```
CLASS TransactionException
  ATTRIBUTE reason AS STRING
  CONSTRUCTOR(message)
    reason <- message
  END CONSTRUCTOR
END CLASS

CLASS AccountFrozenException INHERITS TransactionException
  CONSTRUCTOR(message)
    CALL TransactionException(message)
  END CONSTRUCTOR
END CLASS

CLASS DailyLimitExceededException INHERITS TransactionException
  ATTRIBUTE excessAmount AS REAL
  CONSTRUCTOR(message, excess)
    CALL TransactionException(message)
    excessAmount <- excess
  END CONSTRUCTOR
END CLASS

FUNCTION DebitLedger(amount AS REAL, accountFrozen AS BOOLEAN)
  IF accountFrozen = TRUE THEN
    THROW NEW AccountFrozenException("Account is frozen")
  END IF
  IF amount > 50000 THEN
    THROW NEW DailyLimitExceededException(
      "Daily withdrawal limit exceeded", amount - 50000)
  END IF
  OUTPUT "Ledger debited: $", amount
END FUNCTION

FUNCTION AuditTransactionWrapper(amount, accountFrozen)
  TRY
    CALL DebitLedger(amount, accountFrozen)
  CATCH (e AS TransactionException)
    OUTPUT "SECURITY AUDIT: ", e.reason
    RETHROW
  END TRY
END FUNCTION

MAIN
  TRY
    TRY
      CALL AuditTransactionWrapper(1000, TRUE)
```

```
    CATCH (e AS AccountFrozenException)
      OUTPUT "Transaction declined: account is frozen."
    CATCH (e AS DailyLimitExceededException)
      OUTPUT "Transaction declined: daily limit exceeded."
    END TRY
  CATCH (e AS TransactionException)
    OUTPUT "General transaction failure: ", e.reason
  END TRY
END MAIN
```

2. Dry-run trace: $1,000 withdrawal on a frozen account

Step	Control / event	Result
1	Main calls AuditTransactionWrapper(1000, TRUE)	Control enters intermediate layer.
2	Wrapper calls DebitLedger	Low-level function checks account status.
3	Frozen account detected	AccountFrozenException is thrown; DebitLedger exits.
4	Stack unwinding	Control returns to the wrapper's matching catch.
5	Audit catch executes	Security audit message is written.
6	RETHROW	The same AccountFrozenException continues to Main.
7	Main catch matching	AccountFrozenException handler executes before base handler.

```
SECURITY AUDIT: Account is frozen
Transaction declined: account is frozen.
```

Question 3: Polymorphic Game Engine with Virtual Destructors

Topic: Virtual Functions, Virtual Destructors, Polymorphic Object Lifecycles

1. Formal structured pseudo-code

```
ABSTRACT CLASS GameObject
  PUBLIC:
    PURE VIRTUAL METHOD Update(deltaTime AS REAL)
    PURE VIRTUAL METHOD Render()
    VIRTUAL DESTRUCTOR()
END CLASS

CLASS ParticleSystem INHERITS GameObject
  PRIVATE:
    particleCoordinates AS ARRAY OF REAL
    particleCount AS INTEGER
  PUBLIC:
    CONSTRUCTOR(count)
      particleCount <- count
      ALLOCATE particleCoordinates FOR particleCount * 2 REAL VALUES
    END CONSTRUCTOR
    OVERRIDE METHOD Update(deltaTime AS REAL)
      UPDATE particle positions using deltaTime
    END METHOD
    OVERRIDE METHOD Render()
      OUTPUT "Rendering particle system."
    END METHOD
    OVERRIDE DESTRUCTOR()
      OUTPUT "Releasing particle coordinate array."
      DEALLOCATE particleCoordinates
    END DESTRUCTOR
END CLASS

MAIN
  DECLARE sceneEntities AS ARRAY OF GameObject*
  sceneEntities[0] <- NEW ParticleSystem(100)
  sceneEntities[1] <- NEW ParticleSystem(50)
  FOR EACH entity IN sceneEntities
    entity->Update(0.016)
    entity->Render()
  END FOR
  FOR i <- 0 TO LENGTH(sceneEntities) - 1
    DEALLOCATE sceneEntities[i]
  END FOR
END MAIN
```

2. Cleanup trace and destructor order

Step	Operation	Destructor / result
1	DEALLOCATE sceneEntities[0]	Virtual dispatch selects ParticleSystem destructor.

Step	Operation	Destructor / result
2	Derived destructor body	Particle coordinate array is released.
3	Base destructor	GameObject destructor runs automatically after derived cleanup.
4	DEALLOCATE sceneEntities[1]	The same derived-then-base sequence repeats.

A virtual base destructor ensures that deleting through GameObject* destroys the complete derived object. Destruction order is the reverse of construction: derived destructor first, then base destructor.

Question 4: File Stream Pipeline with Stack Unwinding & Catch-All

Topic: Stack Unwinding, Resource Acquisition (RAII), Catch-All Handler

1. Structured pseudo-code

```
CLASS ResourceGuard
  PRIVATE:
    handle AS FILE_HANDLE
  PUBLIC:
    CONSTRUCTOR(fileName)
      handle <- ACQUIRE_FILE_HANDLE(fileName)
      OUTPUT "File handle acquired."
    END CONSTRUCTOR
    DESTRUCTOR()
      RELEASE_FILE_HANDLE(handle)
      OUTPUT "File handle released."
    END DESTRUCTOR
END CLASS

CLASS CRCValidationException
  ATTRIBUTE expectedChecksum AS INTEGER
  ATTRIBUTE calculatedChecksum AS INTEGER
  CONSTRUCTOR(expected, calculated)
    expectedChecksum <- expected
    calculatedChecksum <- calculated
  END CONSTRUCTOR
END CLASS

FUNCTION IngestDataStream(fileName AS STRING)
  CREATE guard AS ResourceGuard(fileName)
  READ data blocks using guard.handle
  calculated <- COMPUTE_CRC(data blocks)
  expected <- READ_EXPECTED_CRC(fileName)
  IF calculated != expected THEN
    THROW NEW CRCValidationException(expected, calculated)
  END IF
  OUTPUT "File ingestion completed."
END FUNCTION

MAIN
  TRY
    CALL IngestDataStream("mission.dat")
  CATCH (e AS CRCValidationException)
    OUTPUT "CRC validation failed."
    OUTPUT "Expected: ", e.expectedChecksum
    OUTPUT "Calculated: ", e.calculatedChecksum
  CATCH (...)
    OUTPUT "Unknown file ingestion error."
  END TRY
END MAIN
```

2. Stack-unwinding sequence

Step	Event	Control / resource state
1	IngestDataStream starts	Local ResourceGuard is constructed; file handle acquired.
2	Data blocks are read	CRC checksum is calculated.
3	Checksum mismatch	CRCValidationException is thrown.
4	Stack unwinding	Local guard is destroyed and releases the file handle.
5	Main catch executes	Specific CRCValidationException handler reports the mismatch.
6	Fallback	catch(...) handles any other exception not matched above.

The local automatic object is destroyed while the exception propagates out of IngestDataStream. Its destructor runs before the matching catch block in Main.

```
File handle acquired.
File handle released.
CRC validation failed.
Expected: 12345
Calculated: 9876
```

Illustrative checksum values are used in the trace because the task does not supply actual file data or checksum values.

Question 5: Scientific Telemetry Calibration with noexcept Contracts

Topic: Specifying Exceptions, noexcept Contracts, Boundary Assertions

1. Structured pseudo-code

```
CLASS OutOfBoundsException
  ATTRIBUTE measuredTemperature AS REAL
  CONSTRUCTOR(value)
    measuredTemperature <- value
  END CONSTRUCTOR
END CLASS

FUNCTION CalibrateThermalSensors(rawVoltage AS REAL)
  RETURNS REAL NOEXCEPT(FALSE)
  temperature <- CONVERT_VOLTAGE_TO_TEMPERATURE(rawVoltage)
  IF temperature < -273.15 OR temperature > 1500.0 THEN
    THROW NEW OutOfBoundsException(temperature)
  END IF
  RETURN temperature
END FUNCTION

FUNCTION CommitTelemetry(finalTemp AS REAL) NOEXCEPT
  TRY
    WRITE finalTemp TO avionics hardware bus
    OUTPUT "Telemetry committed."
  CATCH (...)
    OUTPUT "Commit error suppressed inside noexcept function."
  END TRY
END FUNCTION

MAIN
  TRY
    temperature <- CalibrateThermalSensors(faultyVoltage)
    CALL CommitTelemetry(temperature)
  CATCH (e AS OutOfBoundsException)
    OUTPUT "Calibration rejected: temperature out of bounds."
    OUTPUT "Measured temperature: ", e.measuredTemperature
  CATCH (...)
    OUTPUT "Unexpected telemetry error."
  END TRY
END MAIN
```

2. Trace: faulty reading of -300.0°C

Step	Operation	Result / state
1	Main calls CalibrateThermalSensors	Calibration computes -300.0°C.
2	Boundary validation	-300.0 is below -273.15°C; OutOfBoundsException is thrown.
3	Exception propagation	Calibration function exits; CommitTelemetry is not called.
4	Main catch matching	OutOfBoundsException handler catches the exception.
5	Report	Main displays the invalid measured temperature.

```
Calibration rejected: temperature out of bounds.
Measured temperature: -300.0 C
```

CommitTelemetry is declared noexcept and catches all exceptions internally. In this trace it is not reached because calibration throws first. The historical dynamic exception specification syntax is not used; noexcept is the modern C++ contract.